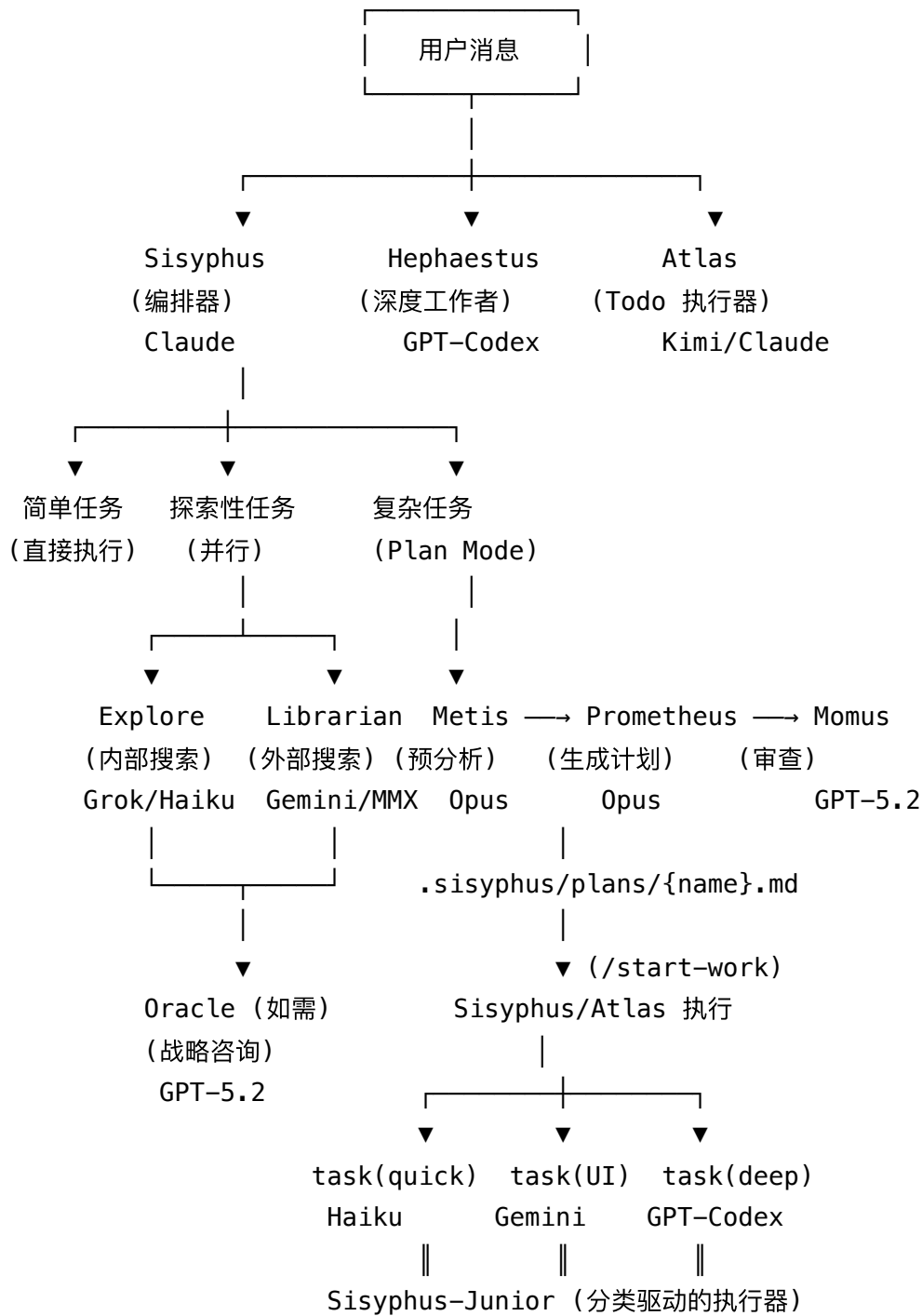


多 Agent 编排原理

编排架构总览

oh-my-opencode 的多 Agent 编排是一个 **分层委派系统**，核心思想是顶层编排器（Sisyphus/Hephaestus/Atlas）通过 `task()` 工具将工作分解委派给专业子代理。



核心调度工具 — task()

入口

源文件: `src/tools/delegate-task/tools.ts` (257 行)

```

// src/tools/delegate-task/tools.ts - task() 工具工厂函数
export function createDelegateTask(options: DelegateTaskToolOptions): ToolDefinition {
  const { userCategories } = options
  const allCategories = mergeCategories(userCategories)
  const categoryNames = Object.keys(allCategories)

  const availableCategories: AvailableCategory[] = options.availableCategories
  ?? Object.entries(allCategories).map(([name, categoryConfig]) => {
    const userDesc = userCategories?.[name]?.description
    const builtinDesc = CATEGORY_DESCRIPTIONS[name]
    return { name, description: userDesc || builtinDesc || "General tasks", model: ca
  })

  return tool({
    description: `Spawn agent task with category-based or direct agent selection.
    ⚠️ CRITICAL: You MUST provide EITHER category OR subagent_type. Omitting BOTH will
    args: {
      load_skills: tool.schema.array(tool.schema.string())
        .describe("Skill names to inject. REQUIRED - pass [] if no skills needed."),
      category: tool.schema.string().optional()
        .describe(`Task category. Available: ${categoryNames.join(", ")}\`),
      subagent_type: tool.schema.string().optional()
        .describe("Direct agent invocation (explore, librarian, oracle, etc.)"),
      prompt: tool.schema.string().describe("Detailed task prompt. MUST be in English."
      description: tool.schema.string().describe("Short task description (3-5 words)",
      run_in_background: tool.schema.boolean().describe("true=async, false=sync wait",
      session_id: tool.schema.string().optional().describe("Session continuation ID",
      command: tool.schema.string().optional().describe("Trigger command",
    },
    async execute(args, toolContext) {
      // 1. resolveSkillContent(args.load_skills)
      // 2. resolveParentContext(ctx, client)
      // 3. session_id 续传 → continuation
      // 4. category 路径 → resolveCategoryExecution()
      // 5. subagent_type 路径 → resolveSubagentExecution()
      // 6. executeBackgroundTask() / executeSyncTask()
    },
  })
}

```

调度决策流程



源文件导航

文件	职责
tools.ts	task() 工具定义和主执行逻辑
types.ts	类型定义 (DelegateTaskArgs, ToolContextWithMetadata)

文件	职责
executor.ts	执行器导出聚合
category-resolver.ts	分类解析 → 模型 + prompt
subagent-resolver.ts	Agent 解析 → 验证 + 模型
model-selection.ts	模型选择管线
prompt-builder.ts	系统 prompt 构建
background-task.ts	后台任务执行
sync-task.ts	同步任务执行
sync-continuation.ts	同步会话续传
background-continuation.ts	后台会话续传
unstable-agent-task.ts	不稳定模型特殊处理
sisyphus-junior-agent.ts	Sisyphus-Junior 名称定义
constants.ts	分类定义、prompt append、描述

两种执行模式

Background（异步）

源文件: src/tools/delegate-task/background-task.ts (98 行)

```
// src/tools/delegate-task/background-task.ts
export async function executeBackgroundTask(
  args: DelegateTaskArgs,
  ctx: ToolContextWithMetadata,
  executorCtx: ExecutorContext,
  parentContext: ParentContext,
  agentToUse: string,
  categoryModel: { providerID: string; modelID: string; variant?: string } | undefined,
  systemContent: string | undefined,
  fallbackChain?: FallbackEntry[],
): Promise<string> {
  const { manager } = executorCtx

  // 通过 manager.launch() 启动后台任务
  // 等待 session 创建 (polling with timeout)
  // 注册 SessionCategoryRegistry
  // storeToolMetadata (title, sessionId, ...)
  // 返回 "Task launched in background.\n\nTask ID: ..."
}
```

使用场景： Explore/Librarian 搜索、不紧急的后台工作

Sync (同步)

源文件: src/tools/delegate-task/sync-task.ts (165 行)

```
// src/tools/delegate-task/sync-task.ts
export async function executeSyncTask(
  args: DelegateTaskArgs,
  ctx: ToolContextWithMetadata,
  executorCtx: ExecutorContext,
  parentContext: ParentContext,
  agentToUse: string,
  categoryModel: { providerID: string; modelID: string; variant?: string } | undefined,
  systemContent: string | undefined,
  modelInfo?: ModelFallbackInfo,
  fallbackChain?: FallbackEntry[],
  deps: SyncTaskDeps = syncTaskDeps // DI for testing
): Promise<string> {
  const { client, directory, onSyncSessionCreated } = executorCtx

  // 1. 创建同步 session (client based)
  // 2. 注册到 subagentSessions / syncSubagentSessions
  // 3. 设置 session agent / fallback chain / category registry
  // 4. 调用 onSyncSessionCreated 回调
  // 5. toast manager 注册任务
  // 6. 执行 prompt → 等待结果
  // 7. 返回格式化的结果 (duration, session_id, output)
}
```

使用场景：需要结果的实现任务、Oracle 重要咨询

并发控制

BackgroundManager

src/features/background-agent/manager.ts — 核心状态管理器

```

class BackgroundManager {
  private tasks: Map<string, BackgroundTask>           // 所有任务
  private notifications: Map<string, BackgroundTask[]> // 完成通知
  private queuesByKey: Map<string, QueueItem[]>        // 按并发 key 排队
  private concurrencyManager: ConcurrencyManager       // 并发控制

  async launch(input: LaunchInput): Promise<BackgroundTask>
  private async processKey(key: string): Promise<void>
  private async startTask(item: QueueItem): Promise<void>
}

```

ConcurrencyManager

src/features/background-agent/concurrency.ts

```

class ConcurrencyManager {
  getConcurrencyLimit(model): number
    // 优先级: modelConcurrency > providerConcurrency > defaultConcurrency > 5

  async acquire(model): Promise<void>
    // 信号量机制: 超限则排队等待

  release(model): void
    // 释放 → 移交给下一个等待者
}

```

任务生命周期

pending → running → completed | error | cancelled | interrupt

模型选择管线

三步解析

Step 1: Intent (意图层)

- └─ uiSelectedModel → 用户在 UI 选择的模型
- └─ userModel → 配置文件覆盖的模型

Step 2: Constraints (约束层)

- └─ availableModels → 当前可用的模型集合

Step 3: Policy (策略层)

- └─ fallbackChain → Agent/Category 专属降级链
- └─ systemDefaultModel → 系统默认模型

分类模型解析

源文件: src/tools/delegate-task/category-resolver.ts (192 行)

```
// src/tools/delegate-task/category-resolver.ts
export interface CategoryResolutionResult {
  agentToUse: string
  categoryModel: { providerID: string; modelID: string; variant?: string } | undefined
  categoryPromptAppend: string | undefined
  maxPromptTokens?: number
  modelInfo: ModelFallbackInfo | undefined
  actualModel: string | undefined
  isUnstableAgent: boolean
  fallbackChain?: FallbackEntry[]
  error?: string
}

export async function resolveCategoryExecution(
  args: DelegateTaskArgs,
  executorCtx: ExecutorContext,
  inheritedModel: string | undefined,
  systemDefaultModel: string | undefined
): Promise<CategoryResolutionResult> {
  const { client, userCategories, sisypusJuniorModel } = executorCtx
  const availableModels = await getAvailableModelsForDelegateTask(client)
  const categoryName = args.category!
  const enabledCategories = mergeCategories(userCategories)

  // 解析分类配置: 用户分类 > sisypus-junior.model > 分类默认
  const resolved = resolveCategoryConfig(categoryName, {
    userCategories, inheritedModel, systemDefaultModel, availableModels
  })
  // → 返回 { agentToUse, categoryModel, categoryPromptAppend, isUnstableAgent }
}
```

Agent 模型解析

源文件: src/tools/delegate-task/subagent-resolver.ts (143 行)

```
// src/tools/delegate-task/subagent-resolver.ts
export async function resolveSubagentExecution(
  args: DelegateTaskArgs,
  executorCtx: ExecutorContext,
  parentAgent: string | undefined,
  categoryExamples: string
): Promise<{
  agentToUse: string;
  categoryModel: { providerID: string; modelID: string; variant?: string } | undefined;
  fallbackChain?: FallbackEntry[];
  error?: string
}> {
  const { client, agentOverrides } = executorCtx

  // 校验: 空名称 → error
  // 校验: 不允许直接调用 Sisyphus-Junior → error (应通过 category)
  // 校验: plan-family agent 不能委托给 plan-family → error
  // 从 client.app.agents() 获取所有 agent
  // 过滤出 callableAgents (mode !== "primary")
  // 匹配 agent 名称 (支持 display name 解析)
  // 解析 model override / fallback chain → 返回
}
```

系统 Prompt 构建

源文件: src/tools/delegate-task/prompt-builder.ts (55 行, 完整源码)

```
// src/tools/delegate-task/prompt-builder.ts - 完整源码
import { buildPlanAgentSystemPrepend, isPlanAgent } from "../constants"
import { buildSystemContentWithTokenLimit } from "../token-limiter"

const FREE_OR_LOCAL_PROMPT_TOKEN_LIMIT = 24000

function usesFreeOrLocalModel(
  model: { providerID: string; modelID: string; variant?: string } | undefined
): boolean {
  if (!model) return false
  const provider = model.providerID.toLowerCase()
  const modelId = model.modelID.toLowerCase()
  return provider.includes("local")
    || provider === "ollama"
    || provider === "lmstudio"
    || modelId.includes("free")
}

/**
 * Build the system content to inject into the agent prompt.
 * Combines skill content, category prompt append, and plan agent system prepend.
 */
export function buildSystemContent(input: BuildSystemContentInput): string | undefined
const {
  skillContent, skillContents, categoryPromptAppend,
  agentsContext, maxPromptTokens, model, agentName,
  availableCategories, availableSkills,
} = input

const planAgentPrepend = isPlanAgent(agentName)
  ? buildPlanAgentSystemPrepend(availableCategories, availableSkills)
  : ""

const effectiveMaxPromptTokens = maxPromptTokens
  ?? (usesFreeOrLocalModel(model) ? FREE_OR_LOCAL_PROMPT_TOKEN_LIMIT : undefined)

return buildSystemContentWithTokenLimit(
  { skillContent, skillContents, categoryPromptAppend,
    agentsContext: agentsContext ?? planAgentPrepend, planAgentPrepend },
  effectiveMaxPromptTokens
)
}
```

Session 续传机制

当父 Agent 对子代理的结果不满意时，通过 `session_id` 续传：

```
// 首次调用
result = task(category="quick", prompt="Fix type error in auth.ts", ...)
// result 包含 session_id: "ses_abc123"

// 续传修复
result = task(session_id="ses_abc123", prompt="Fix: Line 42 still has TypeError", ...)
```

优势：

- 子代理保留完整会话上下文
- 不重复文件读取、探索、设置
- 节省 70%+ tokens
- 子代理知道已尝试/已学到的内容

防护机制

Plan Family 防递归

```
if (isPlanFamily(agentName) && isPlanFamily(parentAgent)) {
  return error("You are a plan-family agent. You cannot delegate to other plan-family a
}
```

Primary Agent 不可委派

```
if (isPrimaryAgent) {
  return error(`Cannot call primary agent "${name}" via task. Primary agents are top-le
}
```

不稳定 Agent 特殊处理

Gemini、MiniMax 模型被标记为 `isUnstableAgent`，使用 `executeUnstableAgentTask()` 执行，配合 `unstable-agent-babysitter` hook 进行崩溃恢复。

8 个内置分类

源文件: `src/tools/delegate-task/constants.ts` (580 行)

```
// src/tools/delegate-task/constants.ts — 核心分类定义
export const DEFAULT_CATEGORIES: Record<string, CategoryConfig> = {
  "visual-engineering": { model: "google/gemini-3.1-pro", variant: "high" },
  ultrabrain:           { model: "openai/gpt-5.3-codex", variant: "xhigh" },
  deep:                 { model: "openai/gpt-5.3-codex", variant: "medium" },
  artistry:             { model: "google/gemini-3.1-pro", variant: "high" },
  quick:                { model: "anthropic/claude-haiku-4-5" },
  "unspecified-low":    { model: "anthropic/claude-sonnet-4-6" },
  "unspecified-high":   { model: "anthropic/claude-opus-4-6", variant: "max" },
  writing:               { model: "kimi-for-coding/k2p5" },
}

export const CATEGORY_DESCRIPTIONS: Record<string, string> = {
  "visual-engineering": "Frontend, UI/UX, design, styling, animation",
  ultrabrain: "Use ONLY for genuinely hard, logic-heavy tasks...",
  deep: "Goal-oriented autonomous problem-solving...",
  artistry: "Complex problem-solving with unconventional, creative approaches...",
  quick: "Trivial tasks – single file changes, typo fixes...",
  "unspecified-low": "Tasks that don't fit other categories, low effort required",
  "unspecified-high": "Tasks that don't fit other categories, high effort required",
  writing: "Documentation, prose, technical writing",
}

// 每个分类都有专属的 prompt append
export const CATEGORY_PROMPT_APPENDS: Record<string, string> = {
  "visual-engineering": VISUAL_CATEGORY_PROMPT_APPEND,
  ultrabrain: ULTRABRAIN_CATEGORY_PROMPT_APPEND,
  deep: DEEP_CATEGORY_PROMPT_APPEND,
  artistry: ARTISTRY_CATEGORY_PROMPT_APPEND,
  quick: QUICK_CATEGORY_PROMPT_APPEND,
  "unspecified-low": UNSPECIFIED_LOW_CATEGORY_PROMPT_APPEND,
  "unspecified-high": UNSPECIFIED_HIGH_CATEGORY_PROMPT_APPEND,
  writing: WRITING_CATEGORY_PROMPT_APPEND,
}
```

源文件: src/shared/model-requirements.ts (170 行) — 分类/Agent 模型 fallback 链

```
// src/shared/model-requirements.ts - 核心类型定义
export type FallbackEntry = {
  providers: string[] // ["anthropic", "github-copilot"]
  model: string // "claude-opus-4-6"
  variant?: string // "max"
}

export type ModelRequirement = {
  fallbackChain: FallbackEntry[]
  variant?: string
  requiresModel?: string
  requiresAnyModel?: boolean
  requiresProvider?: string[]
}

export const CATEGORY_MODEL_REQUIREMENTS: Record<string, ModelRequirement> = {
  "visual-engineering": { fallbackChain: [/* gemini-3.1-pro → glm-5 → claude-opus-4-6 */
  ultrabrain: { fallbackChain: [/* gpt-5.3-codex → gemini-3.1-pro → claude-op
  deep: { fallbackChain: [/* gpt-5.3-codex → claude-opus-4-6 → gemini-3
  quick: { fallbackChain: [/* claude-haiku-4-5 → gemini-3-flash → gpt-5-
  // ...
}
```

分类	默认模型	Prompt 风格	适用场景
visual-engineering	gemini-3.1-pro (high)	设计优先、大胆审美、非传统布局	前端 UI/UX
ultrabrain	gpt-5.3-codex (xhigh)	深度逻辑推理、先搜索现有模式	复杂架构
deep	gpt-5.3-codex (medium)	目标驱动自主执行（5-15 分钟探索）	自主问题解决
artistry	gemini-3.1-pro (high)	突破常规、激进实验	高创意任务
quick	claude-haiku-4-5	快速聚焦 + 低能力模型警告	小改动
unspecified-low	claude-sonnet-4-6	中等投入	通用中等任务
unspecified-high	claude-opus-4-6 (max)	高投入	通用高强度任务

分类	默认模型	Prompt 风格	适用场景
writing	kimi-k2.5	反 AI 腔调、自然写作	文档写作

动态 Prompt 构建器

src/agents/dynamic-agent-prompt-builder.ts 提供了一组函数，根据可用 Agent/Tool/Skill/Category 动态构建 Sisyphus/Hephaestus 的 prompt 节：

函数	生成内容
buildKeyTriggersSection()	Phase 0 触发条件列表
buildToolSelectionTable()	工具/Agent 选择优先级表
buildExploreSection()	Explore 使用指南
buildLibrarianSection()	Librarian 使用指南
buildDelegationTable()	域 → Agent 委派映射表
buildCategorySkillsDelegationGuide()	分类 + 技能完整指南
buildOracleSection()	Oracle 使用协议
buildHardBlocksSection()	硬性禁止列表
buildAntiPatternsSection()	反模式列表
buildDeepParallelSection()	深度并行指导（非 Claude）
buildUltraworkSection()	Ultrawork 资源概览

Agent 注册流程

src/agents/builtin-agents.ts — createBuiltinAgents()：

1. 读取可用模型 (fetchAvailableModels)
2. 合并分类配置 (mergeCategories)
3. 构建可用技能列表 (buildAvailableSkills)
4. 收集通用 Agent 配置 (collectPendingBuiltinAgents)
 - 遍历 agentSources (oracle, explore, librarian, metis, momus, ...)
 - 检查 disabledAgents 和 agentOverrides
 - 应用模型解析管线
 - 收集 availableAgents 元数据
5. 解析自定义 Agent 摘要 (parseRegisteredAgentSummaries)
6. 特殊处理:
 - Sisypheus: 注入 availableAgents + Skills + Categories
 - Hephaestus: 注入 availableAgents + Skills + Categories
 - Atlas: 注入 OrchestratorContext
7. 返回 Record<string, AgentConfig>